

ZÁVĚREČNÁ STUDIJNÍ PRÁCE

dokumentace

Mage Survivor: hra vytvořena v Godot Enginu



Autor: Ondřej Jasník
Obor: 18-20-M/01 INFORMAČNÍ TECHNOLOGIE
se zaměřením na počítačové sítě a programování
Třída: IT4
Školní rok: 2025/26

Prohlášení

Prohlašuji, že jsem závěrečnou práci vypracoval samostatně a uvedl veškeré použité informační zdroje.

Souhlasím, aby tato studijní práce byla použita k výukovým a prezentačním účelům na Střední průmyslové a umělecké škole v Opavě, Praskova 399/8.

V Opavě 31. 12. 2025

.....
Podpis autora

Abstrakt

Tato závěrečná práce dokumentuje proces vývoje akční hry Mage Survivor v herním engine Godot 4. Hra je založena na populárním žánru „survivor“, kde hráč ovládá mága, který musí přežít vlny nepřátel a postupně posilovat své schopnosti. Práce popisuje implementaci různých herních mechanik včetně systému legendárních předmětů, ekonomického systému s truhlami, zajímavých vizuálních efektů a uživatelského rozhraní. Pro implementaci byl využit programovací jazyk GDScript nativně podporovaný engine Godot. Součástí práce je i analýza použitých technologií a popis jednotlivých herních systémů s ukázkami zdrojového kódu.

Klíčová slova

Godot Engine, GDScript, vývoj her, survivor hra, herní mechaniky, legendární předměty, vizuální efekty

Obsah

Úvod	2
1 Godot Engine a GDScript	3
1.1 O enginu Godot	3
1.2 Programovací jazyk GDScript	3
2 Základní herní mechaniky	5
2.1 Architektura projektu	5
2.2 Hráčská postava	5
2.3 Systém nepřátel	6
2.4 Systém zkušeností a levelování	7
3 Systém legendárních předmětů	8
3.1 Fire Boots – Ohnivá stopa	9
3.2 Lightning Aura – Elektrická aura	10
3.3 Frost Ring – Zmrazovací prsten	11
3.4 Shadow Cloak – Stínový plášť	12
3.5 Star Crown – Automatické střelby	13
4 Ekonomický systém a truhly	14
4.1 Systém zlatých mincí	14
4.2 Chest systém	14
4.3 Vzácnost předmětů	16
5 Vizuální efekty	17
5.1 Particle systémy	17
5.2 Sprite sheet animace	18
6 Uživatelské rozhraní	19
6.1 HUD (Head-Up Display)	19
6.2 Main menu	21
Závěr	23

Úvod

Už od malička jsem rád hrával počítačové hry a vždycky mě zajímalo, kolik práce je za tím a jak těžké musí být vytvořit něco, co vypadá tak jednoduše. Proto jsem se rozhodl, že si to zkusím sám a jako téma závěrečné práce jsem si vybral vývoj vlastní hry.

Inspiraci jsem našel v nedávno vydané hře Megabonk. Tato hra nemá nijak výjimečnou grafiku ani extra komplikované mechaniky, ale přesto mě bavila. Řekl jsem si, že bych mohl vytvořit něco podobného, ale ve 2D formě. Začal jsem jednoduše s postavou, která se pohybuje pomocí virtuálního joysticku. Postupně jsem přidával další prvky – životy, animace, nepřátele se spawnerem, systém obtížnosti, levelování, ekonomiku s truhlami a legendární předměty.

Během vývoje jsem čelil různým technickým výzvám. Musel jsem optimalizovat particle systém pro Fire Boots, synchronizovat animace, správně nastavit layerování objektů pomocí z-indexu a vytvořit plynulou shuffle animaci v chest UI. Většinu grafických assetů jsem našel na internetu nebo použil AI generátory, protože grafika není moje silná stránka.

Práce je rozdělena do šesti kapitol popisujících Godot Engine, základní herní mechaniky, legendární předměty, ekonomický systém, vizuální efekty a uživatelské rozhraní.

1 Godot Engine a GDScript

1.1 O enginu Godot

Godot Engine je open-source herní engine vyvíjený komunitou vývojářů. První verze byla vydána v roce 2014 a od té doby prošel značným vývojem. Verze 4.0, která byla použita v tomto projektu, přinesla zásadní vylepšení ve výkonu, nový renderer a lepší podporu pro 3D hry.

Hlavní výhody Godot Engine:

- **Open-source** – Engine je zcela zdarma a jeho zdrojový kód je volně dostupný
- **Malá velikost** – Celý editor má méně než 100 MB
- **Node systém** – Intuitivní hierarchické uspořádání herních objektů
- **Scény jako stavební bloky** – Každý herní objekt je uložen jako samostatná scéna
- **GDScript** – Jednoduchý skriptovací jazyk podobný Pythonu
- **Multiplatformní** – Podpora exportu na Windows, Linux, macOS, Android, iOS a web

1.2 Programovací jazyk GDScript

GDScript je jazyk, který Godot používá pro programování her. Když jsem s ním začínal, hned mi připomněl Python – používá stejné odsazování místo složených závorek a celkově je dost intuitivní. To mi hodně pomohlo, protože jsem nemusel trávit spoustu času učením se úplně nové syntaxe. Zároveň má GDScript spoustu funkcí přímo pro tvorbu her, takže jsem nemusel všechno programovat od nuly.

1.2.1 Základní syntaxe

GDScript používá odsazení (indentaci) místo složených závorek pro definování bloků kódu. Proměnné se deklarují pomocí klíčového slova `var` a funkce pomocí `func`. Hlavní funkce jsou `_ready()` pro inicializaci a `_process(delta)` pro aktualizaci každý snímek.

1.2.2 Node systém

V Godot Engine je vše organizováno do stromové struktury nodů (uzlů). Každý node má specifickou funkci – například `Sprite2D` zobrazuje 2D obrázek, `CharacterBody2D` poskytuje fyziku pro postavy, a `Timer` spouští časované události. Pro přístup k child nodům (potomkům) se používá symbol `$` nebo funkce `get_node()`.

1.2.3 Signály

Signály jsou mechanismus pro komunikaci mezi objekty bez přímé závislosti. Objekt může vyslat signál a jiné objekty se mohou k tomuto signálu připojit a reagovat na něj.

```
1 # Definice vlastního signálu
2 signal health_changed(new_health)
3
4 func take_damage(amount):
5     health -= amount
6     # Vyslání signálu
7     health_changed.emit(health)
8
9 func _ready():
10     # Připojení k signálu tlačítka
11     $Button.pressed.connect(_on_button_pressed)
12
13 func _on_button_pressed():
14     print("Tlačítko bylo stisknuto")
```

Kód 1.1: Použití signálů

2 Základní herní mechaniky

2.1 Architektura projektu

Projekt Mage Survivor je strukturován do několika hlavních složek:

- `scenes/` – Obsahuje všechny herní scény (`.tscn` soubory)
- `scripts/` – GDScript soubory s herní logikou
- `assets/` – Grafické assety (textury, sprite sheety)
- `audio/` – Zvukové efekty a hudba

Hlavní scény projektu:

- `main.tscn` – Hlavní herní scéna
- `mage.tscn` – Hráčova postava
- `enemy.tscn` – Nepřítel
- `ui.tscn` – Herní HUD (Head-Up Display)
- `main_menu.tscn` – Hlavní menu

2.2 Hráčská postava

Hráčská postava je implementována jako `CharacterBody2D`, což je speciální typ nodu poskytující vestavěnou podporu pro pohyb a kolize.

2.2.1 Systém pohybu

Hra byla primárně navržena pro mobilní zařízení s ovládáním pomocí virtuálního joysticku. Pro testování na PC byla přidána alternativní podpora klávesnice (šipky). Pohyb je zpracován v metodě `_physics_process()`:


```
1 func _physics_process(delta):
2     # Získání vstupu od hráče - funguje jak pro klavesnici, tak pro joystick
3     # Input.get_vector() automaticky kombinuje vstupy do jednoho vektoru
4     var input_direction = Input.get_vector(
5         "move_left", "move_right",
6         "move_up", "move_down"
7     )
8
9     # Normalizace je důležitá - bez ní by pohyb diagonálně byl rychlejší
10    # Např. pohyb doprava (1,0) + nahoru (0,1) = (1,1) má délku 1.41
11    # Po normalizaci bude délka vždy 1.0, takže rychlost je konstantní
12    if input_direction.length() > 0:
13        input_direction = input_direction.normalized()
14
15    # Vynásobí směr rychlostí - move_speed je nastavena na 300
16    velocity = input_direction * move_speed
17
18    # move_and_slide() se postará o kolize a hladký pohyb
19    # Toto jsem musel změnit z move_and_collide(), protože to lépe funguje
20    move_and_slide()
```

Kód 2.1: Implementace pohybu hráče

2.2.2 Zdraví a smrt

Systém zdraví sleduje aktuální a maximální HP hráče. Při dosžení nuly se spustí sekvence smrti s game over menu. Funkce `take_damage()` kontroluje také Shadow Cloak pro možnost vyhnutí se útoku.

2.3 Systém nepřátel

Nepřátelé jsou vytvářeni pomocí spawneru, který postupně zvyšuje jejich počet a sílu podle uplynulého času.

2.3.1 AI nepřátel

Každý nepřítel využívá `NavigationAgent2D` pro pathfinding (hledání cesty) k hráči:

```
1 func _physics_process(delta):
```

```
2     if not player or is_dead:
3         return
4
5     # Nastavení cílové pozice
6     navigation_agent.target_position = player.global_position
7
8     # Získání směru k dalšímu bodu na cestě
9     var next_path_position = navigation_agent.get_next_path_position()
10    var direction = (next_path_position - global_position).normalized()
11
12    # Pohyb nepřítele
13    velocity = direction * move_speed
14    move_and_slide()
15
16    # Kontrola dosahu pro útok
17    var distance = global_position.distance_to(player.global_position)
18    if distance < attack_range:
19        _attack_player()
```

Kód 2.2: AI nepřítele

2.3.2 Spawner systém

Aby hra postupně gradovala a nebyla pořád stejná, musel jsem vytvořit systém, který dynamicky zvyšuje obtížnost. Spawner se stará o vytváření nepřátel na náhodných pozicích mimo obrazovku. Začíná pomalu – každé 2 sekundy vytvoří 3 nepřátele. Ale čím déle hráč přežívá, tím rychlejší a početnější jsou vlny.

2.4 Systém zkušeností a levelování

Hráč získává zkušenosti za zabití nepřátel. Po dosažení určitého množství XP se zvýší úroveň a objeví se menu pro výběr vylepšení. Systém používá exponenciální nárůst požadovaných zkušeností pro každý další level.

3 Systém legendárních předmětů

Jednou z klíčových mechanik hry je systém legendárních předmětů, které poskytují hráči unikátní schopnosti. Bylo implementováno celkem pět legendárních předmětů, každý s vlastním efektem. Každý předmět je reprezentován boolean proměnnou a má vlastní logiku, která se aktualizuje každý snímek:

```
1 # Legendární předměty
2 var has_fireboots: bool = false
3 var has_lightning_aura: bool = false
4 var has_frost_ring: bool = false
5 var has_shadow_cloak: bool = false
6 var has_star_crown: bool = false
7
8 # Timery pro jednotlivé efekty
9 var lightning_aura_timer: float = 0.0
10 var star_crown_timer: float = 0.0
11 var fire_trail_timer: float = 0.0
12 var frost_ring_timer: float = 0.0
13
14 func _updateLegendaryItems(delta):
15     if has_fireboots:
16         _updateFireTrail(delta)
17     if has_lightning_aura:
18         _updateLightningAura(delta)
19     if has_frost_ring:
20         _updateFrostRing(delta)
21     if has_star_crown:
22         _updateStarCrown(delta)
```

Kód 3.1: Struktura legendárních předmětů

3.1 Fire Boots – Ohnivá stopa

Fire Boots jsou jeden z mých oblíbených předmětů v hře. Zanechávají za hráčem ohnivou stopu, která nejen dobře vypadá, ale taky poškozují nepřátele. Každých 0.15 sekundy se vytvoří nová skupina 5-8 ohnivých částic. Musel jsem experimentovat s timingem, protože moc často to vypadalo přehnané a moc pomalu to vypadalo divně.

```
1 var fire_frames = [  
2     preload("res://assets/fire_walk/fire_0.png"),  
3     preload("res://assets/fire_walk/fire_1.png"),  
4     preload("res://assets/fire_walk/fire_2.png"),  
5     preload("res://assets/fire_walk/fire_3.png")  
6 ]  
7  
8 func _update_fire_trail(delta):  
9     fire_trail_timer += delta  
10  
11     # Spawn ohnivé částice každých 0.15 sekundy  
12     if fire_trail_timer >= 0.15:  
13         fire_trail_timer = 0.0  
14         _spawn_fire_trail()  
15  
16 func _spawn_fire_trail():  
17     # Spawn 5-8 náhodných částic  
18     var particle_count = randi_range(5, 8)  
19  
20     for i in range(particle_count):  
21         var fire_sprite = AnimatedSprite2D.new()  
22  
23         # Vytvoření SpriteFrames animace  
24         var sprite_frames = SpriteFrames.new()  
25         sprite_frames.add_animation("burn")  
26         sprite_frames.set_animation_speed("burn", 12)  
27  
28         for frame_idx in range(fire_frames.size()):  
29             sprite_frames.add_frame("burn", fire_frames[frame_idx], frame_idx)  
30  
31         fire_sprite.sprite_frames = sprite_frames  
32         fire_sprite.animation = "burn"  
33         fire_sprite.play()  
34  
35         # Náhodná pozice kolem hráče  
36         var offset = Vector2(randf_range(-20, 20), randf_range(-20, 20))  
37         fire_sprite.global_position = global_position + offset
```

```
38
39     # Náhodná velikost a rotace
40     var random_scale = randf_range(0.3, 0.7)
41     fire_sprite.scale = Vector2(random_scale, random_scale)
42     fire_sprite.rotation = randf_range(0, TAU)
43
44     # Z-index pod hráčem
45     fire_sprite.z_index = 9
46
47     get_parent().add_child(fire_sprite)
48
49     # Animace částice
50     _animate_fire_particle(fire_sprite)
```

Kód 3.2: Fire Boots implementace

Každá ohnivá částice pak má vlastní animaci - postupně mizí (fade-out), pohybuje se nahoru a do stran, a zmenšuje se. Po dokončení animace se automaticky smaže pomocí `queue_free()`.



Obrázek 3.1: Fire Boots - ohnivá stopa za hráčem

3.2 Lightning Aura – Elektrická aura

Lightning Aura automaticky útočí na nejbližší nepřátele elektrickými blesky každé 2 sekundy. Musel jsem najít způsob, jak vybrat nejbližšího nepřítele v dosahu 300 pixelů a

pak mu způsobit damage:

```
1 func _update_lightning_aura(delta):
2     lightning_aura_timer += delta
3
4     if lightning_aura_timer >= 2.0:
5         lightning_aura_timer = 0.0
6         _cast_lightning()
7
8 func _cast_lightning():
9     var enemies = get_tree().get_nodes_in_group("enemies")
10    if enemies.is_empty():
11        return
12
13    # Najít nejbližšího nepřítele
14    var closest_enemy = null
15    var closest_distance = 999999.0
16
17    for enemy in enemies:
18        if not is_instance_valid(enemy):
19            continue
20        var distance = global_position.distance_to(enemy.global_position)
21        if distance < closest_distance and distance < 300:
22            closest_distance = distance
23            closest_enemy = enemy
24
25    if closest_enemy:
26        # Způsobení poškození
27        closest_enemy.take_damage(30)
28        print("Lightning Aura zasáhla nepřítele za 30 damage")
```

Kód 3.3: Lightning Aura implementace

3.3 Frost Ring – Zmrazovací prsten

Frost Ring zpomaluje všechny nepřátele v okolí hráče na 50% jejich rychlosti. Každých 0.5 sekundy kontroluje, kteří nepřátelé jsou v dosahu 120 pixelů, a ty zpomalí. Když nepřítel opustí dosah, rychlost se mu vrátí zpátky:

```
1 var frozen_enemies: Array = []
2 var frost_ring_radius: float = 120.0
3
```

```
4 func _update_frost_ring(delta):
5     frost_ring_timer += delta
6
7     if frost_ring_timer >= 0.5:
8         frost_ring_timer = 0.0
9         _apply_frost_ring()
10
11 func _apply_frost_ring():
12     var enemies = get_tree().get_nodes_in_group("enemies")
13     var currently_frozen = []
14
15     for enemy in enemies:
16         if not is_instance_valid(enemy):
17             continue
18
19         var distance = global_position.distance_to(enemy.global_position)
20
21         if distance < frost_ring_radius:
22             # Nepřítel je v dosahu - zmrazit
23             if not frozen_enemies.has(enemy):
24                 enemy.original_speed = enemy.move_speed
25                 enemy.move_speed = enemy.original_speed * 0.5
26                 currently_frozen.append(enemy)
27             elif frozen_enemies.has(enemy):
28                 # Nepřítel opustil dosah - obnovit rychlost
29                 if "original_speed" in enemy:
30                     enemy.move_speed = enemy.original_speed
31
32     frozen_enemies = currently_frozen
```

Kód 3.4: Frost Ring implementace

3.4 Shadow Cloak – Stínový plášť

Shadow Cloak poskytuje hráči 20% šanci vyhnout se útoku. Kontrola se provádí při příjmu poškození:

```
1 var dodge_chance: float = 0.0
2
3 func take_damage(amount: int):
4     # Kontrola dodge
```

```
5     if has_shadow_cloak and randf() < dodge_chance:
6         print("Útok byl vyhnut díky Shadow Cloak!")
7         return
8
9     # Normalni poskozeni
10    current_hp -= amount
11    if current_hp <= 0:
12        die()
```

Kód 3.5: Shadow Cloak implementace

3.5 Star Crown – Automatické střelby

Star Crown automaticky vystřeluje 3 projektily náhodným směrem kolem hráče každou sekundu. Každý projektil létí jiným náhodným směrem pomocí výpočtu úhlu přes TAU konstantu, což vytváří zajímavý efekt pokrytí celé oblasti.

4 Ekonomický systém a truhly

4.1 Systém zlatých mincí

Zlato jsem do hry přidal, abych dal hráči další důvod zabíjet nepřátele kromě jen získávání zkušeností. Za každého zabitého nepřítele dostane hráč nějaké zlato, které pak může použít na otevírání truhel. Je to docela jednoduchý systém, ale funguje dobře:

```
1 var gold: int = 0
2
3 func add_gold(amount: int):
4     gold += amount
5     print("Získáno ", amount, " zlata! Celkem: ", gold)
```

Kód 4.1: Gold systém

4.2 Chest systém

Truhly se objevují náhodně na mapě a obsahují náhodné předměty. Cena otevření truhel se postupně zvyšuje.

4.2.1 Globální počítadlo truhel

Truhly mají postupně se zvyšující cenu, aby hráč nemohl otevírat jednu truhlu za druhou bez rozmyslu. První truhla stojí 50 zlata, každá další se združí o 25 zlata. Použil jsem static proměnnou, která si pamatuje, kolik truhel už hráč otevřel, a podle toho se vypočítá aktuální cena:

```
1 static var chests_opened: int = 0
2 var base_cost: int = 50
3 var cost_increment: int = 25
4
5 func get_chest_cost() -> int:
```

```
6     return base_cost + (chests_opened * cost_increment)
7
8 func open_chest():
9     var cost = get_chest_cost()
10
11     if player.gold < cost:
12         print("Nedostatek zlata!")
13         return
14
15     player.gold -= cost
16     chests_opened += 1
17
18     # Zobrazit chest UI
19     var chest_ui = chest_ui_scene.instantiate()
20     get_tree().root.add_child(chest_ui)
```

Kód 4.2: Progrese ceny truhel

4.2.2 Shuffle animace

Po otevření truhly se spustí animace, která zobrazuje náhodné předměty před odhalením finálního itemu:

```
1 var shuffle_duration: float = 0.08
2 var shuffle_iterations: int = 15
3 var current_iteration: int = 0
4
5 func _play_shuffle_animation():
6     current_iteration = 0
7
8     for i in range(shuffle_iterations):
9         # Zobrazit nahodny item
10         var random_item = _get_random_item()
11         _display_item(random_item)
12
13         # Cekani pred dalsim itemem
14         await get_tree().create_timer(shuffle_duration).timeout
15         current_iteration += 1
16
17     # Zobrazit finalni item
18     _display_item(final_item)
19     _enable_buttons()
```

Kód 4.3: Shuffle animace v chest UI

4.3 Vzácnost předmětů

Každý předmět v truhlách má určitou vzácnost. Legendární předměty se objevují jen v 5% případů, čímž jsou opravdu cenné. Rare itemy mají 25% šanci a běžné (common) itemy tvoří zbytek. Musel jsem to tak nastavit, aby hráč měl motivaci otevírat více truhel a zároveň aby legendárky byly opravdu vzácné:

```
1 enum ItemRarity { COMMON, RARE, LEGENDARY }
2
3 func _get_random_item() -> ItemType:
4     var roll = randf() * 100.0
5
6     # 5% legendary, 25% rare, 70% common
7     if roll < 5.0:
8         var legendaries = [ItemType.FIREBOOTS, ItemType.LIGHTNING_AURA,
9                             ItemType.FROST_RING, ItemType.SHADOW_CLOAK,
10                            ItemType.STAR_CROWN]
11         return legendaries[randi() % legendaries.size()]
12     elif roll < 30.0:
13         var rares = [ItemType.BLUE_POTION, ItemType.LIGHTNING_BOLT,
14                     ItemType.RUBY, ItemType.SHARP_DAGGER]
15         return rares[randi() % rares.size()]
16     else:
17         var commons = [ItemType.LUCKY_DICE, ItemType.RED_APPLE,
18                        ItemType.IRON_SHIELD, ItemType.BOOTS]
19         return commons[randi() % commons.size()]
```

Kód 4.4: Systém vzácnosti

5 Vizuální efekty

5.1 Particle systémy

Hra využívá vlastní particle systém vytvořený pomocí AnimatedSprite2D nodů. Toto řešení bylo zvoleno z důvodu větší kontroly nad animacemi a možnosti použít PNG sekvence.

5.1.1 Z-index layering

Pro správné vizuální zobrazení je důležité nastavit správné hodnoty z-indexu:

- **-1**: Tilemap (mapa)
- **4-9**: Efekty (ohnivé stopy, exploze)
- **10**: Hráč
- **11**: Nepřátelé
- **20+**: UI elementy

```
1 func _ready():
2     # Hráč má z-index 10
3     z_index = 10
4
5 func _spawn_fire_trail():
6     var fire_sprite = AnimatedSprite2D.new()
7     # Oheň pod hráčem
8     fire_sprite.z_index = 9
9     get_parent().add_child(fire_sprite)
```

Kód 5.1: Nastavení z-indexu

5.2 Sprite sheet animace

Pro animace tlačítek a UI elementů jsou použity sprite sheety – jeden obrázek obsahující více framů animace.

5.2.1 AtlasTexture systém

Godot používá AtlasTexture pro definování jednotlivých framů ze sprite sheetu. Pro pause button byla použita 32x16 textura rozdělená na dva framy:

- Frame 1 (0-16px): Normální stav
- Frame 2 (16-32px): Stisknutý stav

6 Uživatelské rozhraní

6.1 HUD (Head-Up Display)

HUD zobrazuje důležité informace během hry: zdraví, level, zlato, počet zabití a čas přežití.



Obrázek 6.1: HUD - zdravotní lišta, level, zlato a pause tlačítko

6.1.1 Zdravotní lišta

```
1 func _update_health_bar():
2     if not player or not is_instance_valid(player):
3         return
4
5     var display_hp = player.current_hp
6     if display_hp < 0:
7         display_hp = 0
8
9     # Výpočet procenta chybějícího zdraví
10    var missing_health_percent = 1.0 - (float(display_hp) / float(player.max_hp)
11    )
12
13    # Aktualizace šířky prázdné části
14    var empty_width = max_bar_width * missing_health_percent
```

```
14     health_bar_empty.size.x = empty_width
15     health_bar_empty.position.x = -141.0 + (max_bar_width - empty_width)
16
17     # Aktualizace textu
18     health_text.text = str(display_hp) + " / " + str(player.max_hp)
```

Kód 6.1: Aktualizace zdravotní lišty

6.1.2 Pause menu

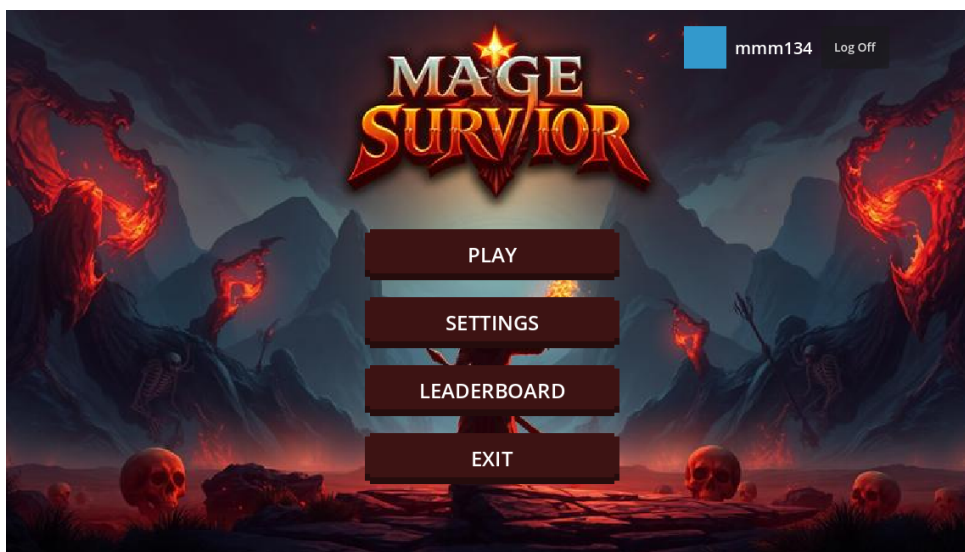
Pause menu zastavuje hru pomocí nastavení `Engine.time_scale` na 0 a zobrazuje statistiky hráče:

```
1 func show_pause_menu():  
2     visible = true  
3     Engine.time_scale = 0.0  
4     update_stats()  
5  
6 func hide_pause_menu():  
7     visible = false  
8     Engine.time_scale = 1.0  
9  
10 func _input(event):  
11     if event.is_action_pressed("ui_cancel"): # ESC  
12         if visible:  
13             hide_pause_menu()  
14         else:  
15             show_pause_menu()
```

Kód 6.2: Pause menu funkce

6.2 Main menu

Hlavní menu obsahuje pozadí s loading screen obrázkem a custom tlačítka.



Obrázek 6.2: Hlavní menu hry

6.2.1 TextureButton s labelem

Pro estetická tlačítka byla použita kombinace TextureButton a Label. Každé tlačítko má přiřazenou texturu a Label potomek pro zobrazení textu, který je vystředěný pomocí anchor systému Godotu.

Závěr

V rámci této závěrečné práce byl vytvořen kompletní funkční prototyp akční hry Mage Survivor v herním engineu Godot 4. Hra obsahuje všechny základní mechaniky žánru survivor – vlny nepřátel, progresivní obtížnost, levelování a upgrade systém.

Hlavními přínosy práce jsou:

- Implementace komplexního systému legendárních předmětů s pěti unikátními efekty
- Vytvoření ekonomického systému s progresivní cenou truhel
- Pokročilé vizuální efekty pomocí animovaných sprite částic
- Intuitivní uživatelské rozhraní s HUD a menu systémy
- Čistý a modulární kód v GDScriptu usnadňující budoucí rozšiřování

Možná budoucí vylepšení

Projekt má potenciál pro další rozšíření:

- Přidání dalších legendárních předmětů a jejich kombinací
- Implementace více typů nepřátel s různými schopnostmi
- Systém achievementů a statistik
- Různé mapy s unikátními prostředími
- Zvukové efekty a hudba

Seznam použitelných informačních zdrojů

- [1] *Godot Tutorial Video* [online]. YouTube [cit. 2026-01-04]. Dostupné z: https://www.youtube.com/watch?v=rckLxmH_fK0
- [2] *Your First 2D Game - Player Scene* [online]. Godot Engine Documentation [cit. 2026-01-04]. Dostupné z: https://docs.godotengine.org/en/stable/getting_started/first_2d_game/02.player_scene.html
- [3] *ChatGPT* [online]. OpenAI [cit. 2026-01-04]. Dostupné z: <https://chat.openai.com/>
- [4] *Godot Tutorial Video* [online]. YouTube [cit. 2026-01-04]. Dostupné z: https://www.youtube.com/watch?v=7ZAF_fn3V0c
- [5] *Godot Tutorial Video* [online]. YouTube [cit. 2026-01-04]. Dostupné z: <https://www.youtube.com/watch?v=GiZuWjXmvcc>
- [6] *r/godot - Godot Engine Community* [online]. Reddit [cit. 2026-01-04]. Dostupné z: <https://www.reddit.com/r/godot/>
- [7] *CraftPix - Game Assets and Sprites* [online]. CraftPix [cit. 2026-01-04]. Dostupné z: <https://craftpix.net/categorys/sprites/>
- [8] *Text to Image AI Generator* [online]. DeepAI [cit. 2026-01-04]. Dostupné z: <https://deepai.org/machine-learning-model/text2im>
- [9] *Google AI Studio* [online]. Google [cit. 2026-01-04]. Dostupné z: <https://aistudio.google.com/apps>
- [10] *Pixel Art Village* [online]. Pixel Art Village [cit. 2026-01-04]. Dostupné z: <https://pixelartvillage.com/>